

Software Engineering Lab - Coursework

Lab 4: VibeCoding - LLM Pair Programming & Word Scramble Repair

Student Name: Sunil Shindolkar

SRN & Section: PES1UG25CS848 | Section L (Semester 5)

Assigned Lab: Lab 4: Word Scramble Repair (Pygame)

AI / LLM Tool Used: ChatGPT (GPT-5.6 Luna)

Conversation Share Link: This ChatGPT conversation

1. Overview & Objectives

The objective of Lab 4 is to use Vibe Coding and AI pair-programming to diagnose and repair a broken Pygame word-scramble game, then incrementally implement the requested enhancement features. The work was completed task-by-task with testing after each change.

Project: Word Scramble Repair. The game selects a word, scrambles its letters, accepts a player guess, maintains score/feedback, and was extended with hints, a countdown timer, and graphical letter tiles.

2. Task 1: Fix Incorrect Guess Comparison

Problem Analysis: The original `submit_guess()` method compared the player's input against `self.scrambled_word`. Therefore, entering the correctly unscrambled secret word could still be marked wrong.

Prompt Sent to ChatGPT:

I am working on a Pygame Word Scramble game for a software engineering lab. The `submit_guess()` method is incorrectly comparing the entered guess with the scrambled word. Fix it so the player's answer is compared with the actual secret word. Keep the existing score, feedback, and next-round behavior.

Solution Applied: The comparison was changed to use `self.secret_word`.

```
is_correct = (guess == self.secret_word)
```

After this change, a correct unscrambled answer awards one point and advances to the next round.

3. Task 2: Implement Hint Points and Letter Reveal

Problem Analysis: The assignment requires a clickable HINT feature that reveals the next unrevealed letter in the correct position. During implementation, the hint system was designed with a separate hint-points counter so hints do not directly reduce the main score.

Prompt Sent to ChatGPT:

Add a clickable HINT button beside SUBMIT. Start with 4 hint points. When HINT is clicked, consume one hint point and reveal the next unrevealed letter of self.secret_word in its correct position. Do not reduce the main score. A correct answer should increase score by 1 and restore 1 hint point. Hint points must never go below 0.

Solution Applied:

```
self.hint_points = 4
self.revealed_letters = []

def use_hint(self):
    if self.hint_points <= 0:
        self.feedback_msg = "No hints available!"
        self.feedback_color = (240, 80, 80)
        return

    for i in range(len(self.secret_word)):
        if i not in self.revealed_letters:
            self.revealed_letters.append(i)
            self.hint_points -= 1
            self.feedback_msg = f"Hint revealed letter {i + 1}!"
            self.feedback_color = (255, 220, 80)
            return
```

The HINT button was connected inside handle_event(), and the revealed positions were reset when next_round() starts.

4. Task 3: Implement Countdown Timer

Problem Analysis: The game originally had no time limit. A countdown timer was added so that the player has a limited amount of time to solve each round.

Prompt Sent to ChatGPT:

Add a 20-second countdown timer. Reset the timer for every new round. If the timer reaches zero before a correct answer, reveal the secret word, show TIME'S UP!, and automatically move to the next round after a short delay.

Solution Applied:

```
self.time_limit = 20
self.start_time = time.time()
self.time_up = False
self.time_up_time = 0

elapsed_time = time.time() - self.start_time
remaining_time = self.time_limit - elapsed_time

if remaining_time <= 0:
    self.time_up = True
    self.time_up_time = time.time()
    self.feedback_msg = f"TIME'S UP! The word was {self.secret_word}."
    self.feedback_color = (240, 80, 80)
```

5. Task 4: Graphical Scrambled Letter Tiles

Problem Analysis: The original scrambled word was rendered as plain text. The task requires graphical square tiles and allows the player to rearrange letters before submitting.

Prompt Sent to ChatGPT:

Replace the plain scrambled-word text with graphical square tiles. Each tile should contain one scrambled letter. Allow the player to click one tile and then another tile to swap their positions. SUBMIT should use the current tile arrangement when the text input is empty.

Solution Applied:

```
self.tile_letters = []
self.selected_tile = None
self.tile_rects = []

# inside next_round()
self.tile_letters = list(self.scrambled_word)
self.selected_tile = None
self.tile_rects = []

# tile click / swap logic
for i, rect in enumerate(self.tile_rects):
    if rect.collidepoint(event.pos):
        if self.selected_tile is None:
            self.selected_tile = i
        else:
            self.tile_letters[self.selected_tile], self.tile_letters[i] = (
                self.tile_letters[i],
                self.tile_letters[self.selected_tile]
            )
            self.selected_tile = None
        break
```

The render function creates square tile rectangles, draws each letter inside its tile, and highlights the selected tile. This gives the player an interactive way to rearrange the letters.

6. Integrated Submit Behavior

The final submit logic first checks the text input. If the input is empty, it uses the current tile arrangement as the guess. Correct answers increase the score and hint points; incorrect answers show feedback and clear the input.

```
def submit_guess(self):
    guess = self.input_box.text.strip().upper()

    if not guess:
        guess = "".join(self.tile_letters)

    if not guess:
        self.feedback_msg = "Arrange the letters before submitting!"
        self.feedback_color = (240, 170, 50)
        return

    is_correct = (guess == self.secret_word)

    if is_correct:
        self.score += 1
        self.hint_points += 1
        self.feedback_msg = f"CORRECT! '{self.secret_word}' is right."
        self.feedback_color = (80, 230, 110)
        self.next_round()
    else:
        self.feedback_msg = "WRONG GUESS! Try again."
        self.feedback_color = (240, 80, 80)
        self.input_box.clear()
```

7. Testing / Verification

The implementation was checked after each task. The intended final behavior is:

Test	Expected Result
Correct unscrambled answer	Correct message, score +1, next round
Wrong answer	Wrong message and input is cleared
Empty text submission	No crash; tile arrangement is used when available
HINT click	Next unrevealed letter is revealed and one hint point is consumed
No hint points	No hint is revealed and a warning is shown
Timer reaches zero	Secret word is revealed and TIME'S UP! is shown
Tile selection	Selected tile is highlighted
Two tile clicks	The two letters swap positions
New round	Secret word, timer, revealed positions, and tile state reset

8. Deliverables & Artifact Verification

- video-before.mp4: 10-second video showing the original broken behavior before repair.
- video-after.mp4: 10-second video showing the completed Word Scramble game with the requested enhancements.
- Full Codebase: Word Scramble project containing game_engine.py, text_box.py, main.py, and README.md.
- ChatGPT Conversation: Complete LLM debugging/pair-programming history for Lab 4.
- Documented History: This chat-history PDF.

Student: Sunil Shindolkar **SRN:** PES1UG25CS848 **Section:** L